

Paso 24A — separar availability y valor

Primero haría una tabla formal de availability lift para TRAIN.

WITH x AS (

```
SELECT
  label_3m,
```

```
  STACK(
    6,
```

```
    'active_accounts_z',
    active_accounts_z_available_flag,
```

```
    'balance_change',
    CASE
      WHEN balance_change_vs_baseline IS NOT NULL
      THEN 1 ELSE 0
    END,
```

```
    'avg_utilization_change',
    CASE
      WHEN utilization_change_vs_baseline IS NOT NULL
      THEN 1 ELSE 0
    END,
```

```
    'weighted_utilization_change',
    CASE
      WHEN weighted_utilization_change_vs_baseline IS NOT NULL
      THEN 1 ELSE 0
    END,
```

```
    'avg_utilization_z',
    avg_utilization_z_available_flag,
```

```
    'weighted_utilization_z',
    weighted_utilization_z_available_flag
```

```
  ) AS (feature_name, available_flag)
```

```
FROM
  bu_frd.model_behavior_features_selected_2022_2025
```

```
WHERE model_split = 'TRAIN'
```

),

base AS (

```
SELECT
    AVG(CAST(label_3m AS DOUBLE)) AS train_positive_rate
FROM x
```

)

```
SELECT
    feature_name,
    available_flag,

    COUNT(*) AS parties,

    SUM(label_3m) AS cases,

    COUNT(*) - SUM(label_3m) AS controls,

    ROUND(
        100.0 * AVG(CAST(label_3m AS DOUBLE)),
        4
    ) AS positive_rate_pct,

    ROUND(
        AVG(CAST(label_3m AS DOUBLE))
        / MAX(train_positive_rate),
        4
    ) AS lift
```

```
FROM x
CROSS JOIN base
```

```
GROUP BY
    feature_name,
    available_flag
```

```
ORDER BY
    feature_name,
    available_flag DESC;
```

Paso 24B — deciles solo entre valores disponibles

WITH long_features AS (

```
SELECT
```

```

party_id,
label_3m,

STACK(
    7,

    'active_accounts_change',
    CAST(active_accounts_change_vs_baseline AS DOUBLE),

    'active_accounts_z',
    CAST(active_accounts_z_win AS DOUBLE),

    'balance_change',
    CAST(balance_change_vs_baseline AS DOUBLE),

    'avg_utilization_change',
    CAST(utilization_change_vs_baseline AS DOUBLE),

    'weighted_utilization_change',
    CAST(weighted_utilization_change_vs_baseline AS DOUBLE),

    'avg_utilization_z',
    CAST(avg_utilization_z_win AS DOUBLE),

    'weighted_utilization_z',
    CAST(weighted_utilization_z_win AS DOUBLE)

) AS (feature_name, feature_value)

FROM
    bu_frd.model_behavior_features_selected_2022_2025

WHERE model_split = 'TRAIN'
),

available AS (

    SELECT *
    FROM long_features
    WHERE feature_value IS NOT NULL

),

ranked AS (

    SELECT
        *,
        NTILE(10) OVER (

```

```

        PARTITION BY feature_name
        ORDER BY feature_value
    ) AS decile

FROM available
),

base AS (

    SELECT
        feature_name,
        AVG(CAST(label_3m AS DOUBLE))
        AS available_positive_rate

    FROM available

    GROUP BY feature_name
)

SELECT
    r.feature_name,
    r.decile,

    COUNT(*) AS parties,
    SUM(r.label_3m) AS cases,

    MIN(r.feature_value) AS min_value,
    percentile_approx(r.feature_value,0.50)
    AS median_value,
    MAX(r.feature_value) AS max_value,

    ROUND(
        100.0 * AVG(CAST(r.label_3m AS DOUBLE)),
        4
    ) AS positive_rate_pct,

    ROUND(
        AVG(CAST(r.label_3m AS DOUBLE))
        / MAX(b.available_positive_rate),
        4
    ) AS lift_vs_available_population

FROM ranked r

INNER JOIN base b
    ON r.feature_name = b.feature_name

GROUP BY

```

```
r.feature_name,  
r.decile
```

```
ORDER BY  
  r.feature_name,  
  r.decile;
```

Paso 24C — bins para sparse features

```
WITH x AS (
```

```
  SELECT  
    label_3m,
```

```
  CASE  
    WHEN deterioration_acceleration < 0  
      THEN 'NEGATIVE'  
    WHEN deterioration_acceleration = 0  
      THEN 'ZERO'  
    WHEN deterioration_acceleration > 0  
      THEN 'POSITIVE'  
  END AS deterioration_group,
```

```
  CASE  
    WHEN adverse_acceleration < 0  
      THEN 'NEGATIVE'  
    WHEN adverse_acceleration = 0  
      THEN 'ZERO'  
    WHEN adverse_acceleration > 0  
      THEN 'POSITIVE'  
  END AS adverse_group,
```

```
  CASE  
    WHEN past_due_change_vs_baseline < 0  
      THEN 'NEGATIVE'  
    WHEN past_due_change_vs_baseline = 0  
      THEN 'ZERO'  
    WHEN past_due_change_vs_baseline > 0  
      THEN 'POSITIVE'  
    ELSE 'MISSING'  
  END AS past_due_group,
```

```
  CASE  
    WHEN delinquency_share_change_vs_baseline < 0  
      THEN 'NEGATIVE'  
    WHEN delinquency_share_change_vs_baseline = 0  
      THEN 'ZERO'  
    WHEN delinquency_share_change_vs_baseline > 0
```

```

        THEN 'POSITIVE'
      ELSE 'MISSING'
    END AS delinquency_group,

    CASE
      WHEN max_dpd_jump_recent_3m = 0
        THEN '0'
      WHEN max_dpd_jump_recent_3m = 1
        THEN '1'
      WHEN max_dpd_jump_recent_3m >= 2
        THEN '2+'
    END AS dpd_jump_group

  FROM
    bu_frd.model_behavior_features_selected_2022_2025

  WHERE model_split = 'TRAIN'
),

long_sparse AS (

  SELECT
    label_3m,

    STACK(
      5,

      'deterioration_acceleration',
      deterioration_group,

      'adverse_acceleration',
      adverse_group,

      'past_due_change',
      past_due_group,

      'delinquency_share_change',
      delinquency_group,

      'max_dpd_jump_recent',
      dpd_jump_group

    ) AS (feature_name, feature_group)

  FROM x
),

base AS (

```

```

SELECT
    AVG(CAST(label_3m AS DOUBLE))
    AS train_positive_rate
FROM x
)

SELECT
    feature_name,
    feature_group,

    COUNT(*) AS parties,
    SUM(label_3m) AS cases,

    ROUND(
        100.0 * AVG(CAST(label_3m AS DOUBLE)),
        4
    ) AS positive_rate_pct,

    ROUND(
        AVG(CAST(label_3m AS DOUBLE))
        / MAX(train_positive_rate),
        4
    ) AS lift

FROM long_sparse
CROSS JOIN base

GROUP BY
    feature_name,
    feature_group

ORDER BY
    feature_name,
    feature_group;

```